

National Real-Time Payments System

Architectural and Design Considerations

Prepared for: Emily Watson, IT Technical Lead

Prepared by: Rajasekar S

Date: March 2026

1. INTRODUCTION

1.1 About NRTPS

The rapid shift toward digital economies has accelerated the need for national-scale payment infrastructures capable of supporting instant, secure, and highly reliable financial transactions. The proposed **National Real-Time Payments System (NRTPS)** is envisioned as a foundational platform enabling instant peer-to-peer (P2P) and peer-to-merchant (P2M) payments between citizens, merchants, and financial institutions. The system is expected to handle extremely high transaction volumes, support millions of concurrent users, and maintain strict security and regulatory compliance standards.

With approximately **150 million registered users**, **40 million daily active users**, and peak loads exceeding **50,000 transactions per second**, the platform must be designed with strong scalability, resilience, and security principles.

This report outlines the key architectural components, communication mechanisms, potential failure scenarios, mitigation strategies, and strategic trade-offs involved in designing such a system. It also highlights lessons from similar global payment infrastructures and suggests enhancements that could strengthen the platform's long-term impact.

1.2 Architecture Overview

National Real-Time Payments System to enable instant digital payments between citizens, businesses, and financial institutions. The platform aims to become the primary backbone for all peer-to-peer (P2P) and peer-to-merchant (P2M) transactions, replacing cash and traditional delayed banking transfers.

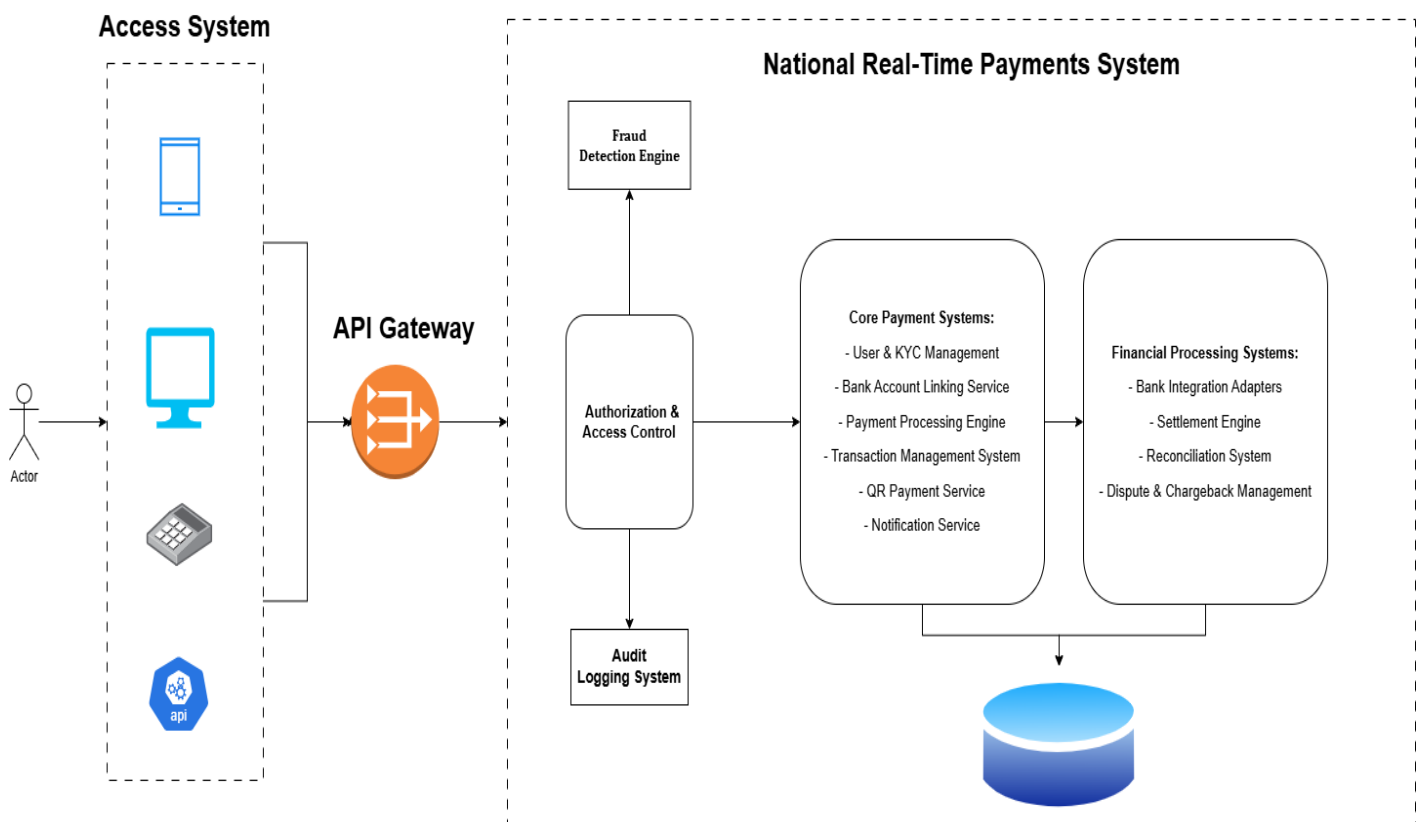


Fig 1 Overall Architecture

1.3 Overall Dataflow Diagram

The architecture diagram Fig2 illustrates the layered design of the National Real-Time Payments System, where multiple user channels such as mobile apps, web applications, merchant POS systems, and third-party integrations access the platform through a centralized API Gateway. The gateway routes requests to core services including Identity & User Management, Payment Processing Engine, and Fraud Detection, which validate and execute transactions in real time. These services interact with the Bank Integration Layer and Settlement & Reconciliation System to complete interbank payments and ensure financial accuracy.

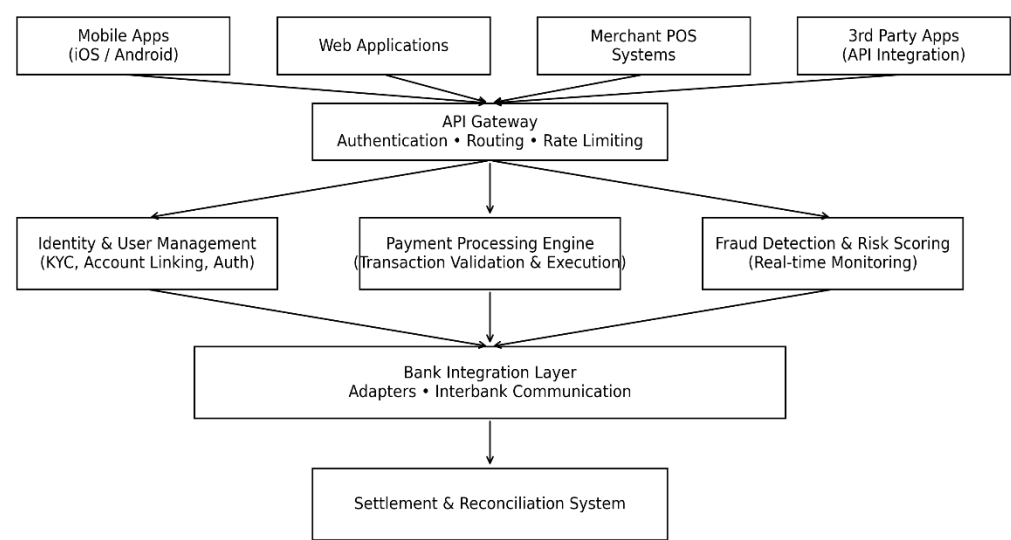


Fig 2. Dataflow Diagram of National Real-Time Payments System

2. CORE SYSTEMS AND SUBSYSTEMS

To deliver the required functionality at national scale, the NRTPS architecture must be composed of multiple distributed subsystems working together.

2.1 Channel Systems

Channel systems act as the entry point for transaction initiation.

Key components include:

- Mobile applications (iOS and Android)
- Web applications
- Merchant POS terminals
- Third-party applications integrated through APIs

These systems interact with the backend through a secure API gateway and enable users to initiate payments, view transaction history, and manage accounts.

2.2 API Gateway and Access Layer

The API Gateway serves as the unified entry point for all incoming requests and performs several critical functions:

- Authentication and authorization
- Rate limiting and traffic control
- Request routing
- API management for external integrations

This layer ensures that all external communication is securely controlled and monitored.

2.3 Identity and User Management System

This subsystem manages user onboarding and authentication. It includes:

- KYC verification services
- Identity management
- Bank account linking
- Multi-factor authentication

Given the regulatory environment, strong identity verification and auditability are essential.

2.4 Payment Processing Engine

The payment processing engine is the core component responsible for executing transactions. It handles:

- Payment validation
- Balance checks
- Payment routing
- Transaction authorization
- Payment execution

This component must be optimized for extremely low latency and high throughput.

2.5 Bank Integration Layer

Since payments involve multiple banks, a dedicated bank integration layer is required to communicate with various banking systems.

Responsibilities include:

- Inter-bank communication
- Payment confirmation
- Settlement coordination
- Bank availability monitoring

Adapters or connectors may be required for different bank systems due to varying integration protocols.

2.6 Settlement and Reconciliation System

This subsystem manages financial settlement between banks.

Functions include:

- Net settlement calculation
- Scheduled settlement processing
- Reconciliation of transactions
- Dispute resolution and chargebacks

This system ensures financial integrity across the ecosystem.

2.7 Fraud Detection and Risk Scoring System

Real-time fraud detection is critical to prevent malicious activities without impacting transaction speed.

Key capabilities include:

- Behavioural analysis
- Transaction risk scoring
- Machine learning-based fraud detection
- Automated fraud alerts

Fraud detection services should operate asynchronously while still providing near real-time risk assessments.

2.8 Monitoring and Operations Platform

Operational visibility is essential for maintaining reliability.

This subsystem provides:

- System health monitoring
- Real-time alerts
- Performance metrics
- Log aggregation
- Incident management dashboards

These capabilities help operations teams detect and respond to issues rapidly.

3. INFORMATION EXCHANGE BETWEEN SYSTEMS

Multiple systems must exchange information continuously to support payment processing.

3.1 Types of Data Exchanged

Key data types include:

- User identity information
- Payment requests
- Bank authorization responses
- Fraud risk scores
- Settlement records
- Audit logs

3.2 Communication Characteristics

Frequency

Payment requests occur continuously, with spikes during national events or salary periods.

Speed

Transaction processing must meet an end-to-end latency requirement of **less than two seconds**.

Volume

The platform must support **50,000+ transactions per second**, with potential spikes up to **10x during peak events**.

Consistency

- Payment confirmations require **strong consistency**
- Reporting and analytics may operate with **eventual consistency**

A combination of **synchronous APIs** and **asynchronous messaging systems** (such as event queues) would enable efficient communication.

4. Potential Failure Points

Given the complexity of the system, multiple failure scenarios must be anticipated.

4.1 Technical Failures

Potential technical failures include:

- Payment processing engine crashes
- Database failures
- Network latency spikes
- API gateway overload

4.2 Integration Failures

Failures may occur when external banking systems experience outages or integration issues.

Examples include:

- Bank API timeouts
- Inconsistent bank response formats
- Delayed bank confirmations

4.3 External Dependency Failures

External dependencies may introduce risk, including:

- Identity verification service failures
 - Cloud infrastructure disruptions
 - Third-party fraud detection service outages
-

5. FAILURE MITIGATION STRATEGIES

To maintain high availability and reliability, several mitigation strategies should be implemented.

5.1 Redundancy and High Availability

- Multi-region deployment
- Active-active data centres
- Redundant service instances

This eliminates single points of failure.

5.2 Circuit Breakers and Retry Mechanisms

Circuit breaker patterns can prevent cascading failures when external services become unavailable. Automatic retry mechanisms can also help recover from transient errors.

5.3 Event-Driven Architecture

Using event-driven communication enables loose coupling between services and improves resilience during peak loads.

5.4 Load Balancing and Auto-Scaling

Infrastructure should automatically scale horizontally during traffic spikes, ensuring stable performance during national events.

6. CONTINGENCY PLANS FOR FAILURES

Even with strong resilience mechanisms, failures may still occur. Therefore, contingency plans must be established.

Manual Reconciliation

In case of system inconsistencies, financial records must be reconciled manually using audit logs and settlement data.

Deferred Settlements

If banks are temporarily unavailable, transactions can be recorded and settled later once bank connectivity is restored.

User Communication

Transparent communication with users is critical. The system should:

- Notify users of delayed transactions
 - Provide clear transaction status updates
 - Offer support channels for dispute resolution
-

7. SYSTEM BEHAVIOUR UNDER DIFFERENT LOAD CONDITIONS

7.1 Normal Daily Load

During regular operations, the system processes transactions with full functionality, including fraud detection, monitoring, and reporting.

7.2 Peak National Events

During high-demand periods such as festivals or salary days, the system must:

- Enable aggressive auto-scaling
- Prioritize transaction processing
- Temporarily reduce non-critical services such as analytics
- Implement request throttling where necessary

This ensures that core payment functions remain stable.

8. ARCHITECTURAL TRADE-OFFS

Designing a real-time payment platform requires balancing competing architectural priorities.

Consistency vs Availability

Financial transactions require strong consistency; however, non-critical services such as analytics can tolerate eventual consistency.

Cost vs Reliability

Highly redundant infrastructure increases cost but is essential for maintaining five-nines availability (99.999%).

Real-Time vs Eventual Processing

Transaction confirmation must be real-time, while settlement reconciliation may occur asynchronously.

Security Depth vs User Experience

Strong security controls such as multi-factor authentication may introduce friction. Therefore, adaptive authentication based on transaction risk may be implemented.

9. GLOBAL IMPLEMENTATIONS AND RECOMMENDED ENHANCEMENTS

Several countries have implemented successful real-time payment platforms, offering valuable lessons.

Global Examples

Examples include:

- Unified Payments Interface (India)
- Faster Payments (United Kingdom)
- PIX (Brazil)

These systems demonstrate the feasibility of national-scale instant payment networks.

Recommended Enhancements

Additional features that could enhance the proposed platform include:

- Open API ecosystem for fintech innovation
- AI-based fraud detection improvements
- Smart payment routing to optimize bank connectivity
- Cross-border payment interoperability

Business Impact and ROI

The implementation of such features could generate significant benefits:

- Increased digital transaction adoption
- Reduced dependency on cash
- Improved financial inclusion
- Lower transaction processing costs
- Opportunities for international payment integration

These advantages strengthen the national financial infrastructure and position the platform for global expansion.

10. CONCLUSION

The National Real-Time Payments System represents a critical infrastructure initiative capable of transforming digital financial transactions at national scale. Designing such a system requires careful attention to scalability, resilience, security, and regulatory compliance.

By implementing a distributed architecture composed of specialized subsystems, robust failure mitigation strategies, and scalable communication patterns, the platform can meet the demanding performance and reliability requirements outlined in the case study.

Furthermore, learning from global implementations and incorporating advanced capabilities such as AI-driven fraud detection and cross-border interoperability will help ensure the long-term success and competitiveness of the system.

With the right architectural approach, the NRTPS platform can serve as a reliable and future-ready backbone for the country's digital economy.